

From one household to many: groups, sync, and chat

Two-Way Match today serves exactly one household from one database. This record decides how it becomes a platform many groups can register on, where each group's data lives, how two phones stay in step, and how a conversation with voice messages fits into the same system — without rewriting what already works.

STATUS ACCEPTED	DATE 1 September 2026	SUPERSEDES TWM-ADR-001 (single household)	RUNTIME TODAY SAP CAP · SQLite · Fly.io fra
TEST BASELINE 478 backend · 249 frontend			

0 · SUMMARY

Seven decisions, one table

Everything below argues for this table. If you read nothing else, read this.

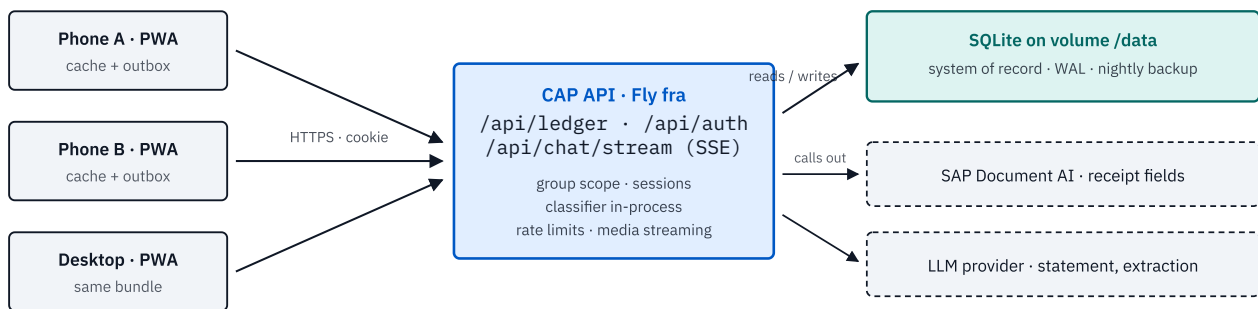
QUESTION	DECISION	REJECTED
Where does data live?	The server is the system of record. Each device keeps a cache and an outbox for offline use.	Local-only on each phone; pure cloud with no cache
How are groups isolated?	One database, a <code>group</code> column on every tenant table, and one handler that narrows every query to the caller's group.	A database per group; a schema per group
Who is a user?	<code>Users</code> log in. <code>People</code> are the roster inside a group. A <code>Membership</code> joins the two, with a role.	Reusing <code>People</code> as accounts
How do messages arrive live?	Server-sent events over the existing HTTP stack; sending stays a normal POST.	WebSockets; a hosted pub/sub vendor
How are voice notes stored?	Recorded in the browser, stored as-is next to receipts, served only through the API. 120 s / 5 MB cap.	Server-side transcoding; public object URLs
Which database engine?	SQLite on the Fly volume now. Postgres when a second machine, a 2 GB file, or sustained chat load arrives.	Postgres from day one
Is chat end-to-end encrypted?	No, deliberately, in v1. TLS in transit, encrypted volume at rest, server never exposes media publicly.	E2EE — it would blind the features the app exists for

1 · CONTEXT

What exists, and what must not break

The running system is a single SAP CAP service at `/api/ledger` in front of a SQLite file on a Fly.io volume in Frankfurt, serving a React PWA. It has fourteen entities, a two-head classifier that runs in-process, an LLM-backed receipt extractor, cookie sessions, and 727 automated tests. It is deployed with a commit-stamped build that refuses to ship a dirty tree.

Two properties of that codebase decide the shape of this design. First, it already has a *query-narrowing* visibility mechanism: unrevealed surprise events are removed from `Events`, `eventTotals` and the calendar by a `before READ` handler that rewrites the query with `not in` — rather than sieving rows afterwards — so counts and pages stay honest, and a hidden id answers `404`, never `403`. Group isolation is that mechanism generalised. Second, the scan flow already has an on-device queue that downscales images and retries uploads. Offline sync is that queue generalised.

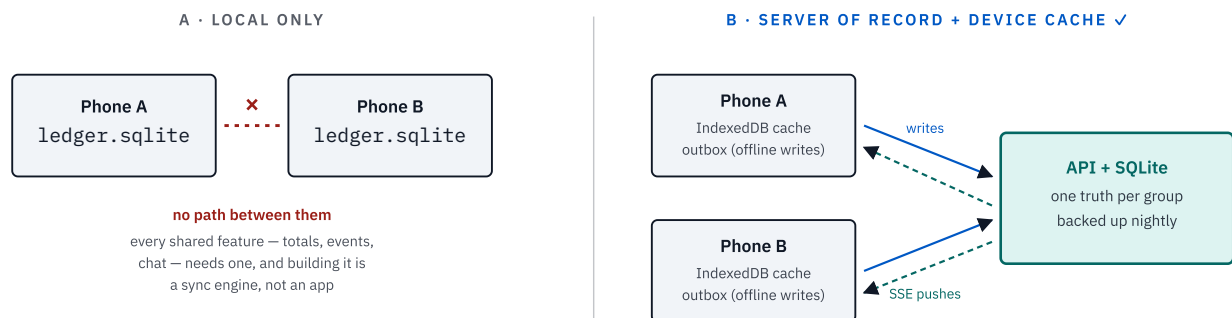


Every device runs the same PWA bundle and talks to exactly one thing: the API. The database, the classifier and the external services sit behind it. Dashed boxes are called out to; nothing calls in.

2 · ADR-2.1 · WHERE DATA LIVES

The server is the system of record; devices keep a working copy

The question the brief asks — local on the phone, or in the cloud — has a short answer once you draw the two phones.



Option A has no edge between the two phones, and every feature this app exists for lives on that edge. Option B adds the server as the one place both phones agree on; the dashed teal arrows are the live pushes that keep each cache warm.

DECISION

Cloud is the system of record; every device keeps a local working copy. The existing CAP server on Fly (Frankfurt) holds the truth. Each device caches what it has read in IndexedDB and queues what it could not send in an outbox. Reads never block on the network when the cache can answer; writes are optimistic and reconciled when the server confirms.

"Local only" is attractive for privacy and for cost, and it is the wrong answer for this product for one reason: the app is *about* two or more people sharing a ledger. Any feature that shows one person what another did — the month total, an event's spend, a message — requires a path between devices. Building that path without a server means device discovery, conflict resolution and merge semantics: a distributed-systems project, not a household app. A server is the cheapest possible implementation of "both phones agree."

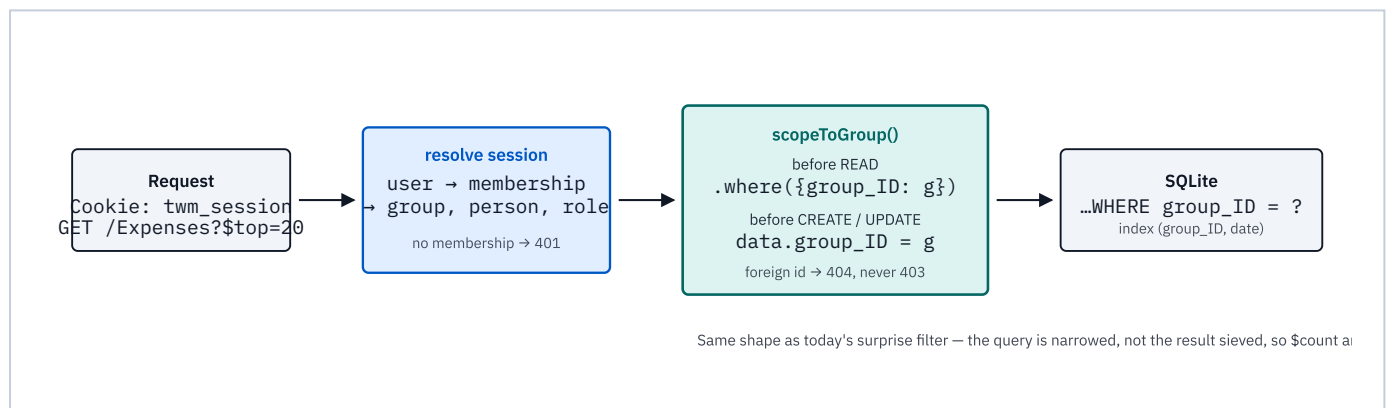
"Pure cloud" is the other trap. The app is an installable PWA that people open on a train and in a restaurant basement. A screen that is blank without signal is a native-app regression. The cache is what makes it feel like an app.

Data residency: Fly `fra` keeps every byte in the EU. For a Swiss household that satisfies the revised FADP; for EU users it satisfies GDPR's transfer rules without further paperwork. The nightly backup artefact must stay in the same region.

3 · ADR-2.2 · TENANCY

One database, a group column, one narrowing handler

Instagram and Facebook are the brief's reference points. What they actually do at the storage layer is not a database per user — it is a tenant key on every row and a discipline that no query runs without it. That discipline is the whole design.



One handler, registered for every tenant entity, is the entire isolation boundary. It runs before CAP builds SQL, so there is no code path that can read another group's rows by accident.

DECISION

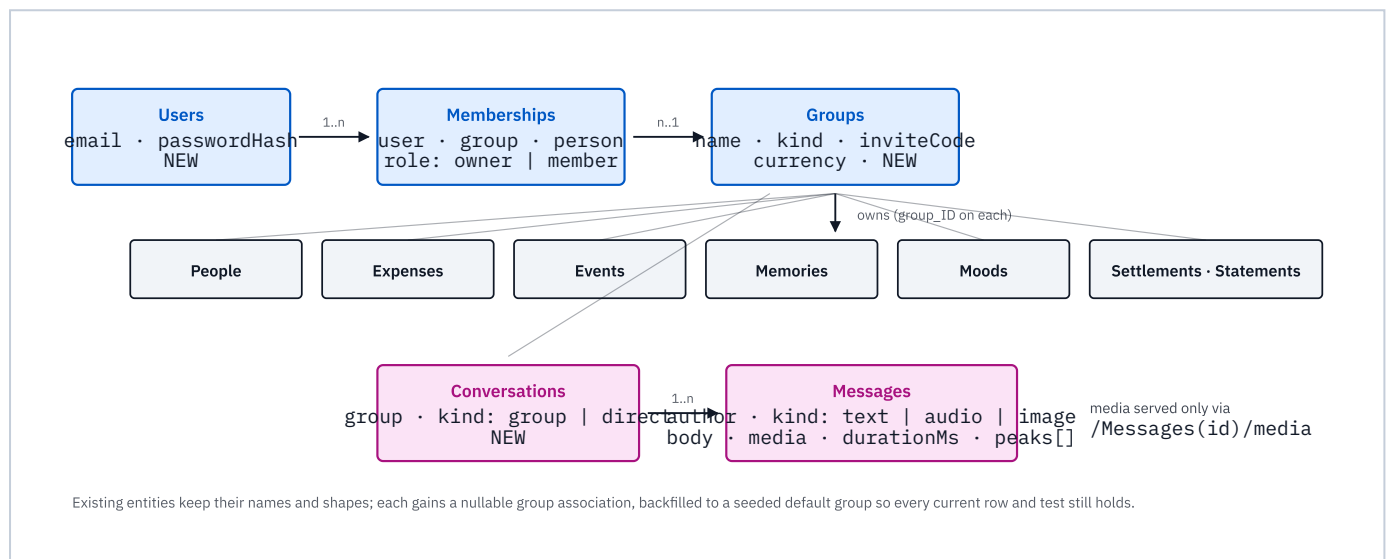
Row-level tenancy. Every entity that belongs to a household gains `group : Association to Groups`. A single `scopeToGroup` handler narrows reads and stamps writes. Cross-group lookups by id return `404` so that the existence of another group's data is itself not disclosed. A composite index `(group_ID, date)` on `Expenses` keeps the Ledger's monthly query flat as groups multiply.

REJECTED · DATABASE PER GROUP

One SQLite file per group on the same volume is tempting — isolation by construction — but it defeats `cds-typex`, breaks the single-connection CAP driver, turns the nightly backup into N backups, and makes any cross-group operation (an admin view, a migration) a loop over files. It buys nothing that the narrowing handler plus tests does not.

Data model

Additive. Nothing that exists is renamed or removed; three entities join the model and one column joins each tenant table.



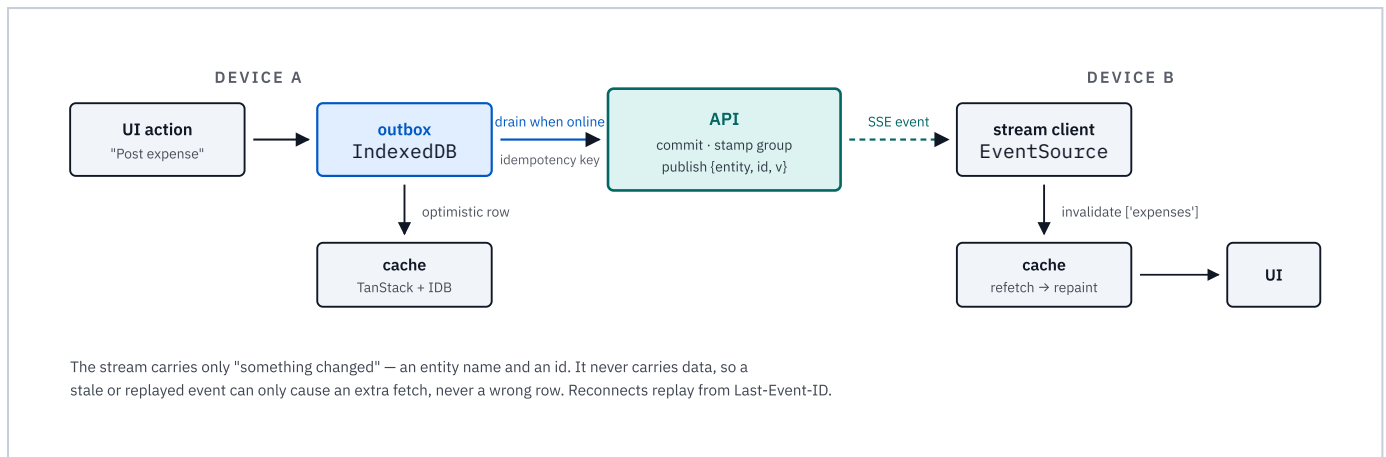
Identity (blue) is separated from the roster: a User can belong to several Groups, and in each one is represented by a Person row. Chat (magenta) hangs off the group like everything else, so the same scoping handler protects it.

```
// db/schema.cds – additive; every existing entity keeps its name and shape
entity Groups : cuid, managed {
  name      : String(120);
  // What kind of household this is. Used for copy and defaults only – never for logic.
  kind      : String(20) enum { couple; household; friends; family; other; };
  currency  : String(3) default 'CHF';
  inviteCode : String(12);           // rotated on demand; joins are one-time
  members   : Composition of many Memberships on members.group = $self;
}
entity Users : cuid, managed {
  email      : String(200) @assert.format: '^[^@]+@[^@]+$';
  passwordHash : String(80);         // bcrypt; never selected by any projection
  displayName : String(100);
  // Optional, self-described, free text. See §6 on why this is not an enum.
  gender     : String(40);
}
entity Memberships : cuid, managed {
  user      : Association to Users;
  group     : Association to Groups;
  person    : Association to People; // this user's roster row inside this group
  role      : String(10) enum { owner; member; };
}
// Tenant aspect, mixed into People, Expenses, Events, Memories, Moods, Settlements,
// Statements, Corrections, Conversations. Nullable during migration, then @mandatory.
aspect tenant { group : Association to Groups; }
```

4 · ADR-2.3 · SYNC

Offline-first without a sync engine

The app never needs two devices to edit the same row at the same moment. That single fact lets it skip CRDTs entirely.



Writes go through an outbox with an idempotency key, so a retried upload cannot post a receipt twice. Other devices learn that a row changed and refetch it; they never receive the row over the stream.

DECISION

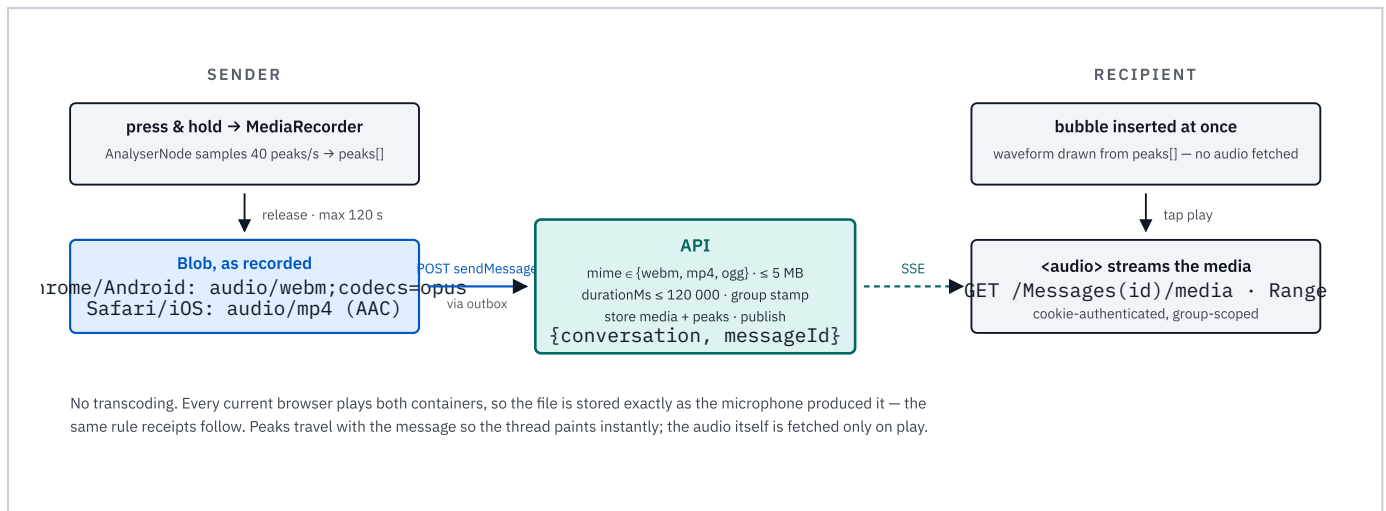
Invalidation, not replication. Devices cache reads and queue writes. The server publishes *notifications* (entity, id, version), never payloads. Conflicts are resolved last-writer-wins on the server with the version number; the UI shows the server's row the moment it arrives. This is enough because two people in one household do not edit the same expense simultaneously in practice, and when they do, the second save wins visibly rather than merging silently.

The scan queue already does two thirds of this: it holds images in IndexedDB, retries, and reconciles against the returned draft. Promoting it to a general outbox is a refactor, not an invention. The idempotency key is a client-generated UUID sent as `Idempotency-Key`; the server stores the last 24 h of keys per group and replays the original response for a duplicate.

5 · ADR-2.4 & 2.5 · CHAT AND VOICE

A conversation is just another tenant entity with a live feed

WhatsApp-like means three things people notice: messages appear without refreshing, a voice note records with one press and plays inline, and the thread survives a bad connection.



The waveform is the trick that makes it feel instant: forty numbers per second captured while recording, stored as a short array, drawn before a single byte of audio is downloaded.

DECISION · REALTIME

Server-sent events on `GET /api/chat/stream`, one connection per open client, subscribed to the caller's group. Sends are ordinary POSTs. On reconnect the browser resends `Last-Event-ID` and the server replays what was missed from the message table. When there is no stream (older browsers, hostile proxies), the thread polls every 15 s — the same code path, slower.

REJECTED · WEBSOCKETS, HOSTED PUB/SUB

Chat delivery is one-directional; the client never needs to push over the socket. WebSockets add an upgrade path through helmet and the CSP, connection-state handling, and a second protocol to test, for no gain here. A hosted service (Pusher, Ably) would move message metadata to a third party outside the EU and add a bill to a household app.

DECISION · VOICE

Record in the browser, store as recorded, serve through the API. `MediaRecorder` with the container the platform prefers; a 120-second and 5 MB ceiling enforced on both sides; media streamed with `Range` support through the cookie-authenticated, group-scoped endpoint — never a public URL. No server-side transcoding: the container matrix is two entries and every current browser plays both.

SCALING NOTE

The SSE subscriber map lives in the Node process. That is correct today — Fly runs exactly one machine (`min_machines_running = 1`, auto-stop off). The moment a second machine exists, the map must move to Postgres `LISTEN/NOTIFY` or Redis, and that is the same trigger that moves the database (§7). The two migrations are one migration.

Groups register; people join by invitation

The brief asks for group types at sign-up. Here is how to have them without storing something the app has no business storing.

The flow

1. **Create account** — email, password, display name. That is a `User`.
2. **Create a group or join one.** Creating asks for a name and a *kind* — *Two of us, A household, Friends, Family*. Joining asks for the invite code the other person was shown.
3. **Your seat in the group** — a `Person` row is created for you with your display name and a colour; a `Membership` binds the three.
4. Document #1, the model, the Ledger and the chat are then *this group's*, and nobody else's.

The two presets the brief names — a man and a woman, or two people of the same sex — both land on *Two of us*. The preset sets the roster size to two and the copy to the couple voice. That is all a preset needs to do.

WHY THE COMPOSITION IS OPTIONAL AND FREE-TEXT, NOT A REQUIRED ENUM

Whether a couple is heterosexual or same-sex is **special-category data** under GDPR Article 9 and the Swiss FADP's "sensitive personal data": it concerns sex life and sexual orientation. Storing it needs explicit consent, a stated purpose, and stronger protection — and this app has *no purpose* for it. Nothing in the ledger, the classifier, the calendar or the chat behaves differently for one couple than another. Making it a required field would collect the most sensitive data in the system for zero product value. So: each person may describe their gender in their own words, or not at all; nothing is inferred from it; it drives copy only, if that. The presets give the brief its sign-up choice; the schema declines to turn that choice into a stored label.

Invitations

An eight-character code, shown once, valid for 72 hours, single use, rotated by the owner on demand. It is enough for a household and it is the mechanism Instagram, Signal and every family app converged on. Links (`/join/CODE`) wrap the same code so a phone can open it from a message.

SQLite until a specific thing happens, then Postgres

A file database is the right engine for a single machine serving hundreds of households. The question is not "is SQLite serious" — it is "what is the trigger to leave it."

TRIGGER	WHY IT MATTERS	MOVE TO
A second Fly machine	SQLite is single-writer on one disk; two machines cannot share the file safely.	Postgres (Fly Postgres, fra) + LISTEN/NOTIFY for SSE fan-out
DB file > 2 GB	Backups and volume snapshots get slow; VACUUM pauses become visible.	Postgres; media to Tigris (Fly's S3), signed URLs still served via the API
Sustained > 10 chat writes/s	Every write takes the single WAL lock; audio inserts are large.	Same move; also chunked audio upload
More than ~1 000 active groups	Not a SQLite limit — an operations one: per-group support, GDPR export/delete, abuse.	Admin service, audit log, per-group export job

Blobs stay in the database until the file passes roughly one gigabyte. Receipts, event photos and voice notes all follow the same path today (stripped, downscaled, stored as `LargeBinary`, streamed through the service), and one path is worth more than an early optimisation. When they move to object storage, the API keeps issuing the URLs — a short-lived signed URL is handed out only after the group check passes — so nothing in the client changes and nothing becomes publicly addressable.

8 · ADR-2.8 · SECURITY

What protects a household's data, and what deliberately does not

- **Sessions** — the existing `httpOnly`, `SameSite=Lax` cookie, HMAC-signed, 7-day expiry, gains a `groupId` claim. Switching groups re-issues it.
- **Authorisation** — `scopeToGroup` on every tenant entity and every action; an id from another group is `404`. This is tested by a suite that creates two groups and proves each cannot see, count, total, or stream the other.
- **Media** — receipts, photos and voice notes are never served from a public path. `Range` requests are supported so audio scrubs, but every byte passes the cookie and the group check.
- **Rate limits** — already on login, scan and statement; added on `sendMessage` (60/min) and media upload (20/min) per user.
- **At rest** — the Fly volume is encrypted; bcrypt for passwords; the LLM key and session secret live only in Fly secrets.
- **Erasure** — deleting a group cascades through compositions and detaches nothing: the whole household's rows go, the backup rotates them out within 30 days. This is the GDPR/FADP right to erasure, implemented as one action.

NOT IN V1 · END-TO-END ENCRYPTION OF CHAT

E2EE would mean the server cannot read messages — and this server's job is to read the household's data: it totals expenses, writes the yearly statement, classifies receipts and answers the calendar. Blinding it to chat while it reads everything else buys an inconsistent promise. If the product ever becomes many strangers rather than households who share a ledger on purpose, revisit with a per-group key held on devices. Say this plainly in the privacy note rather than implying WhatsApp-grade secrecy.

9 · MIGRATION

Four phases, each shippable, none destructive

The constraint from the brief — do not destroy what works — becomes the plan. Every phase leaves 727 tests green and a deployable tree.

0 FOUNDATION	Add <code>Groups</code> , <code>Users</code> , <code>Memberships</code> , <code>Conversations</code> , <code>Messages</code> ; add a <i>nullable group</i> to every tenant entity; seed one default group and backfill every existing row to it. Regenerate types. gate: cds compile · 478 backend tests unchanged · app boots
1 ISOLATION	Session carries <code>groupId</code> . <code>scopeToGroup</code> registered for every tenant entity and action. Registration, invite and join flows. Login page gains "Create a group". Column becomes mandatory. gate: two-group isolation suite · every existing test still passes under the default group
2 CHAT	SSE stream, <code>sendMessage</code> / media endpoint, the thread UI with press-to-record, waveform bubbles, unread counts, and the outbox promoted from the scan queue. gate: message round-trip through two browsers in Playwright · voice note recorded, uploaded, played
3 POLISH	The UI pass: home grid density, consistent card system across pages, motion on the launcher and chat, dark theme parity, the "Payment run" mobile fix, and copy that speaks in the group's voice. gate: phone screenshots of every page reviewed · no horizontal overflow · Lighthouse ≥ 90

Delivery pipeline

Unchanged, because it already refuses the mistakes that matter: `git commit` → CI runs typecheck, both suites and the production build → `npm run deploy` stamps the commit SHA into the bundle and refuses a dirty tree →

Fly builds the image remotely and rolls it out in `fra` → `/health` reports the build it is running. Each phase above is a sequence of such deploys, not a big bang.

TWM-ADR-002 · Two-Way Match · accepted 1 September 2026 · supersedes ADR-001